Aerial Firefighting Vehicle Routing under Coupled Fire Simulation

T. Gupta

Contents

1 Introduction **1**

2 Literature Review **3**
2.1 Fire Propagation 3
2.2 Water Dynamics 4
2.3 Optimisation 5
2.4 Synthesis 5

1 Introduction

Climate change is increasing both the frequency and severity of wildfires by altering temperature, precipitation, and vegetation patterns [**?**], and burnt area in EU countries has repeatedly exceeded historical averages in recent fire seasons [**?**]. Aerial firefighting (AFF) remains one of the primary tools available to suppress wildfires before they escalate, yet the coordination of aircraft fleets during an active fire is still largely dependent on the experience and judgement of human operators. Given the scale, time pressure, and safety stakes involved, there is a clear need for decision-support tools that can compute efficient aircraft routing and suppression strategies fast enough to be usable in live operations.

The Vehicle Routing Problem (VRP) and its variants have long been adapted to disaster and humanitarian relief settings, where capacity limits, multiple depots, and split deliveries are common features of the underlying logistics, as in the Split Delivery VRP [**?**] and drone-based relief distribution under payload and energy constraints [**?**]. **?**] adapt these ideas specifically to wildfire suppression, introducing the Aerial FireFighting Vehicle Routing Problem (AFFVRP): a capacitated, multi-trip VRP with time windows and a hierarchical objective that first minimises time-to-last-drop and then total flight time, using a heterogeneous fleet of tankers and scoopers. **?**] extend this direction with a hierarchical decision-making model for multi-aircraft *formations*, explicitly modelling the water-release process and tactical coordination behaviours that pure routing formulations do not capture. In both cases, however, the fire itself is represented as a static, pre-computed set of drop demands, with no feedback between the suppression actions taken and how the fire subsequently continues to spread.

Physically grounded fire spread models close part of this gap from the other direction. FARSITE [**?**] propagates a fire perimeter using Huygens' principle of wavelet expansion driven by the Rothermel surface fire spread equations, offering high fidelity with respect to fuel, terrain, and weather, but with no native mechanism for representing suppression actions. DEVS-FIRE [**?**] represents the burning area as a grid of discrete-event cells explicitly designed to accommodate suppression inputs, at the cost of the physical detail a continuous, terrain-driven model provides. No existing formulation couples a routing and suppression optimisation directly and iteratively with a physically realistic fire spread simulator.

This dissertation develops an optimisation framework for aerial firefighting that discretises the operational area into a finite grid of cells and a finite set of timesteps, allowing a routing and suppression model to be coupled directly with a fire spread simulator in a rolling-horizon loop. Each grid cell carries one of three states — unburned, burning, or extinguished — and fire propagation, aircraft flight dynamics, and water-drop suppression are all expressed over this shared discretisation. Aircraft are heterogeneous, following [**?**], water drops follow a hexagonal footprint approximated by a normal lateral distribution, and wind is treated stochastically to reflect operational uncertainty. The two objectives of minimising fire spread and minimising time to full extinguishment are retained from [**?**].

At the time of writing, a mixed-integer linear program solved with Gurobi implements this discretised formulation, including fire propagation, water-capacity dynamics for both aircraft types, and linear flight dynamics constraints. A rolling-horizon pipeline automates FARSITE (run inside FlamMap) to close the loop between suppression and continued fire spread. Several extensions are in progress: replacing the current probabilistic extinguishment rule with a deterministic, numerically-derived suppression criterion; allowing aircraft state to carry over continuously across rolling-horizon windows rather than resetting to the depot at each step; introducing a discrete-event simulation layer using salabim [**?**] to validate optimiser output before execution and reduce the computational load on the MILP; and migrating more detailed aircraft flight dynamics into this simulation layer.

The remainder of this dissertation is organised as follows. [PLACEHOLDER — Chapter **??** presents the mathematical formulation of the discretised AFFVRP model.] [PLACEHOLDER — Chapter **??** describes the FARSITE coupling and rolling-horizon methodology.] [PLACEHOLDER — Chapter **??** presents the results and case studies.] Chapter **??** concludes this dissertation and outlines directions for future work.

2

Literature Review

This chapter reviews the literature underpinning the three components of the framework developed in this dissertation: how fire spread is modelled (Section 2.1), how water drops interact with a fire and with aircraft flight (Section 2.2), and how aircraft routing and suppression decisions are formulated as an optimisation problem (Section 2.3). Each section closes by connecting the reviewed material directly to the implementation already in place.

2.1. Fire Propagation

Wildfire spread models fall broadly into two families: continuous, physics- and geometry-based simulators, and discrete, cell-based event simulators.

FARSITE. FARSITE [**?**] simulates two-dimensional fire growth by treating every vertex on the current fire perimeter as the source of an expanding elliptical wavelet, following Huygens' principle; the new perimeter is the outer envelope of all such wavelets after one timestep. At each vertex, local fuel, terrain, and wind data are used to parameterise the wavelet shape via the Rothermel surface fire spread equation,

$$R = \frac{I_R \, \xi \, (1 + \phi_w + \phi_s)}{\rho_b \, \varepsilon \, Q_{ig}}, \tag{2.1}$$

which gives the equilibrium rate of spread as a function of reaction intensity I_R, propagating flux ratio ξ, wind and slope coefficients ϕ_w, ϕ_s, bulk density ρ_b, effective heating number ε, and heat of pre-ignition Q_{ig}. FARSITE additionally captures crown fire transition via Van Wagner thresholds, spotting from torching trees, and time-varying weather and wind streams, and adjusts its own timestep dynamically so that the fastest-moving vertex never exceeds a set distance resolution. Its outputs are fire perimeters together with vertex-level fireline intensity, flame length, and rate of spread. FARSITE has no native representation of suppression: nothing in the model can remove or slow a burning cell in response to an external action.

DEVS. The DEVS approach discretises the terrain into a grid of cells, each an atomic discrete-event model exchanging messages with its eight neighbours. Cells occupy one of six states (unburned, burning, burned, unburned-wet, burning-wet, burnt-wet), and a cell ignites either from an external ignition source exceeding an intensity threshold or from a neighbour transmitting fire with sufficient directional intensity, with local spread shape again assumed elliptical and driven by a Rothermel-based (BEHAVE) implementation. Crucially, DEVS represents suppression natively: water or retardant drops are injected as external events that move a cell into a wet state, either pre-empting ignition or extinguishing an already-burning cell after a stochastic delay. This event-driven, message-passing structure makes DEVS considerably easier to couple with an external decision-making or optimisation layer than a continuous simulator such as FARSITE, at the cost of the physical detail FARSITE provides.

Positioning. The grid used throughout this dissertation adopts a three-state simplification of the DEVS cell model (unburned, burning, extinguished) together with the same Moore-neighbourhood,

message-passing style of propagation, implemented in `grid_utils.py` and the propagation constraints of `optimisation.py`. Rather than replacing FARSITE with this simplified grid, however, the two are run in a rolling-horizon loop: FARSITE (via `farsite_interface.py`) supplies physically grounded fire growth for each window, and its output is parsed back onto the DEVS-style grid on which suppression decisions are then optimised. This combines FARSITE's physical fidelity with the suppression-friendly, decision-coupled structure that motivates the DEVS approach, rather than requiring a choice between the two.

2.2. Water Dynamics

Water release process and flight constraints. ?] model the water release process over a fire scene as three phases — arrival, release, and departure — governed by aircraft speed, water discharge, and climb angle constraints. Maximum speed depends on the aircraft's water state,

$$v_{\max} = \begin{cases} 140\ \text{km/h}, & \text{without water}, \\ 160\ \text{km/h}, & \text{carrying water}, \\ 80\ \text{km/h}, & \text{dropping water}, \end{cases} \tag{2.2}$$

and the maximum bank angle $\phi_{\max}$ is 10° without water and 15° carrying water, giving a turning radius $R = |v|^2/(g \tan \phi)$. During departure, the climb angle γ and true altitude TA must satisfy $\gamma \geq \text{slope}_{\text{terrain}}$ and $TA \geq h_{\min}$ to avoid terrain collision. Two further operational rules are imposed: aircraft may only fly over burning areas while releasing water, and water release itself must occur in level flight.

Drop pattern. The same work approximates the water dispersion footprint as a hexagon, itself approximated by a normal distribution whose width is coupled to aircraft speed and retardant descent velocity. The longitudinal profile is piecewise linear,

$$\lambda(x,h) = \begin{cases} kh^{1.5}\,\dfrac{x}{l_1}, & x < l_1, \\ kh^{1.5}, & l_1 \leq x \leq l_1 + l_2, \\ kh^{1.5}\,\dfrac{2l_1 + l_2 - x}{l_1}, & l_1 + l_2 < x \leq 2l_1 + l_2, \end{cases} \tag{2.3}$$

and the full footprint follows

$$\eta(x,y) = \frac{\lambda}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\frac{y^2}{2\sigma^2}\right), \qquad \sigma = \lambda/6, \tag{2.4}$$

where k scales the maximum discharge width for a given release altitude h, and l_1, l_2 characterise the longitudinal extent of the pattern. Release heights of 10–80 m are reported as typical. The fire area itself is discretised into a set of fire points $FA = \{fp_1^1, \ldots, fp_i^1, \ldots, fp_j^n\}$, each $fp = (x_f, y_f)$.

Positioning. The $\lambda(x,h)$ and $\eta(x,y)$ expressions above are implemented directly and without modification in `drop_pattern.py` (`_lambda` and `eta`), and the recommended grid cell size in `config.py` is tied to σ at the centre of the footprint via `sigma_at_centre`, exactly as suggested by this formulation. The flight-envelope constraints — speed as a function of water state, bank-angle-limited turning radius, and the climb-angle departure safety margin — are *not yet* present in the optimisation model, which currently limits aircraft only by a simplified Manhattan-distance speed bound. Reintroducing this literature's flight dynamics is planned for the discrete-event simulation layer described in Chapter 1, rather than as additional MILP constraints, since continuous-time kinematics of this kind are considerably more natural to express in a process-based simulation than inside a discretised integer program.

2.3. Optimisation

AFFVRP. ?] formulate the Aerial FireFighting Vehicle Routing Problem on a directed graph with three node types: the airfield, a single nearest water body, and fire nodes. Fires requiring more than one drop are decomposed into *subfires*, each a unit-demand node. Two aircraft types are distinguished: scoopers, which refill at the water body and can perform several sequential drops without returning to the airfield, and tankers, which are faster and carry more water but must return to the airfield after each full discharge. The objective is hierarchical — minimise the time of the latest drop, then minimise total flight time — subject to fleet-size limits on both aircraft types. Two simplifying assumptions are made: a subfire is fully extinguished the moment it is reached, and every aircraft returns to the airfield it departed from. The resulting model is a capacitated, multi-trip VRP in which a tanker may serve several subfires of the *same* fire on a single trip if capacity allows, and a scooper's travel time between two subfires is computed via the water body, implicitly encoding the mandatory refill.

Drone relief routing. A structurally related formulation for humanitarian drone relief expresses the energy cost of an arc as a linear function of payload,

$$R_{ij}(w) = \gamma_0 + \gamma w_{ij} + \delta_{ij} \left(\rho_0 + \rho w_{ij}\right), \tag{2.5}$$

where w_{ij} is the payload carried from i to j, γ_0, γ are fixed and payload-dependent take-off/landing energy costs, ρ_0, ρ are the corresponding per-distance flight costs, and δ_{ij} is the distance between i and j. Standard flow conservation is imposed at every node,

$$\sum_{i \in V \setminus \{j\}} x_{ij} = \sum_{i \in V \setminus \{j\}} x_{ji}, \quad \forall j \in V, \tag{2.6}$$

together with a requirement that drones return empty to the depot, that the payload change across an arc equal the demand at the node being left (except at recharging stations, where payload is unchanged), and that payload never exceed the vehicle capacity via a standard big-M linking constraint. Locations may additionally be assigned priority classes (P1 highest to P3 lowest), enforced through progressively tighter time windows on higher-priority nodes.

Positioning. The AFFVRP's heterogeneous-fleet and multi-trip structure is generalised, rather than copied directly, into the grid-based formulation of `optimisation.py`: tanker and scooper water-capacity dynamics (constraints governing $q^k_{k,t}$ and $q^p_{p,t}$) play the same functional role as the AFFVRP's capacity and refuel logic, but are expressed per grid cell and timestep rather than per graph node. The drone relief model's linear energy-cost structure is conceptually mirrored by these same water-load variables, though the current implementation tracks discrete drop units rather than a continuous energy budget — extending q^k, q^p to a continuous, distance-weighted quantity in the style of $R_{ij}(w)$ is a natural refinement. The priority/time-window mechanism is likewise directly analogous to the (currently dormant) time-window constraints inherited from the AFFVRP formulation and retained in the model, which could be reactivated to encode exactly this kind of fire-priority policy.

2.4. Synthesis

Taken together, this review positions the present work as a synthesis rather than a direct extension of any single source: fire spread retains FARSITE's physical fidelity while suppression is decided over a DEVS-style discretised grid; the water-drop footprint is adopted unmodified from the multi-aircraft formation literature, with its flight-dynamics constraints deferred to a future simulation layer; and the routing and capacity structure generalises the AFFVRP and drone-relief literature from a graph of discrete nodes onto the same spatio-temporal grid used for fire propagation and suppression. [PLACEHOLDER — expand this synthesis once the simulation-layer and numerical-suppression extensions described in Chapter 1 are implemented.]